

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING

Title: Implement Array and String in Assembly
Language Programming

Assembly Programming Lab
CSE 3106

Jashore University of Science and Technology

1 Objective(s)

- To understand the use of Array in Assembly Language Program.
- To understand the use of String in Assembly Language Program.

2 Problem analysis

2.1 Array

Arrays can be seen as chains of variables. A text string is an example of a byte array; each character is presented as an ASCII code value (0..255). Here are some array definition examples:

a DB 48h, 65h, 6Ch, 6Ch, 6Fh, 00h

b DB 'Hello', 0

b is an exact copy of the a array, when compiler sees a string inside quotes it automatically converts it to set of bytes. This chart shows a part of the memory where these arrays are declared:

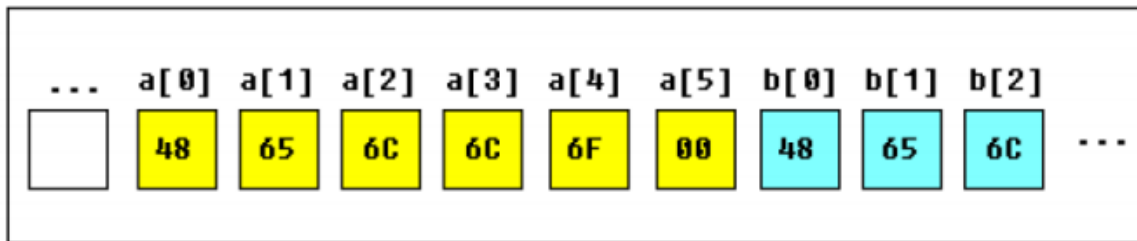


Figure 1: Array Structure

You can access the value of any element in array using square brackets, for example:

```
MOV AL, a[3]
```

You can also use any of the memory index registers BX, SI, DI, BP, for example:

```
MOV SI, 3
```

```
MOV AL, a[SI]
```

If you need to declare a large array you can use DUP operator. The syntax for DUP: For example:

```
c DB 5 DUP(9)
```

c DB 9, 9, 9, 9, 9 ; is an alternative way of declaring:

one more example:

```
d DB 5 DUP(1, 2)
```

d DB 1, 2, 1, 2, 1, 2, 1, 2, 1, 2 ; is an alternative way of declaring:

Of course, you can use DW instead of DB if it's required to keep values larger than 255, or smaller than -128.

DW cannot be used to declare strings!

2.2 String

We can store a string in .data segment. Here we have provided an example :

```
.DATA  
S1 DW 'Hello World'
```

To print a string, we have to write the following instructions:

```
LEA DX, S1  
MOV AH, 09h  
int 21h
```

3 Example of Array and String Code in Assembly

```
1  org 100h
2
3  .DATA ; Data segment starts
4  A db 3, 1, 2, 2, 1 ;1-D array for number
5  B db 00h
6  message db 'Enter the value of N:$' ;1-D array for string
7
8  .CODE ; Code segment starts
9  MAIN PROC
10 mov ax, @DATA
11 mov ds, ax
12
13 xor ax, ax
14 mov si, OFFSET A
15 mov di, OFFSET B
16
17 mov dx, OFFSET message ; Load Effective Address of the message in DX register
18 ; lea dx, message ; (similar meaning that Load Effective Address)
19 mov ah, 09h ;display string function
20 int 21h ;display message
21
22 mov ah, 01h
23 int 21h
24 mov cl, al
25 sub cl, 48 ; to convert the ascii value of 3 to decimal 3
26
27 xor al, al
28
29 Loop_1:
30 add al, [Si]
31 inc Si
32 loop Loop_1
33
34 mov bl, al
35 add bl, 48 ; to convert the ascii value of the output to decimal
36
37 mov ah, 02h
38 mov dl, 0Dh ; Clear Buffer
39 int 21h
40 mov dl, 0Ah ; for newline
41 int 21h
42
43 mov dl, bl
44 int 21h
45
46 mov ah, 4ch
47 int 21h
48
49 MAIN ENDP
50 END MAIN
51 RET
```

4 Sample Input/Output (Compilation, Debugging & Testing)

To derive summation of $3 + 1 + 2 + 2 + 1$ using array A. Here, value of N is given by user where $N=5$ and output 9 will be shown in the output window:

Enter the value of N: 5 9

5 Discussion & Conclusion

Based on the focused objective(s) to understand about array and string in assembly language programming, the additional lab exercise made me more confident towards the fulfilment of the objectives(s).

6 Lab Task (Please implement yourself and show the output to the instructor)

1. Write an assembly language code to print out the elements in an array in reverse order.
2. Write an assembly language code to:
 - a. Take any number of inputs in an array.
 - b. Print out the elements in an array.

[NB: In all program you should use string for input and output message]

7 Lab Exercise (Submit as a report)

- Write an assembly language code to take natural number series as input and as output, show:
 - a. The summation of odd numbers.
 - b. The summation of even numbers.

[NB: In this program you should use string for input and output message]

8 Policy

Copying from internet, classmate, seniors, or from any other source is strongly prohibited. 100% marks will be *deducted* if any such copying is detected.